

claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_ac0cbb29-6bf\agent-ad582a63e7ccee38a.jsonl`  
- SHA-256: `593f3c54865c129e0cc29e7485360c78f2078665d62f98c65eda4b4d8f4e86e1`  
- Source modified: `2026-06-19T15:53:15+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_ac0cbb29-6bf`  
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`
```

Transcript

1. user (2026-06-19T15:52:06.180Z)

You are auditing a badminton rally-detection codebase for DOC/COMMENT STALENESS ? statements that CONTRADICT the current code or data.

HARD RULES:

- Report ONLY factual staleness with CODE EVIDENCE (file:line or symbol) that proves it. Open the cited code and confirm before reporting.
- NEVER invent a fact or a status. If a thing is built-but-default-OFF / unwired / unmeasured, say EXACTLY that ? do not overclaim "in production".
- A "[design]" tag is stale ONLY if the thing is actually BUILT in code (cite the symbol).
- DO NOT propose subjective rewrites, style/tone changes, or improvements. DO NOT relitigate owner-gated DECISIONS (owned-model roadmap, licensing/MIT-Apache stance, product/strategy choices, genuinely-still-design proposals).
- Preserve the document's voice and formatting; make the SMALLEST surgical edit. stale_text MUST be copied verbatim with enough context to be unique.
- IGNORE docs/archives/** (terminal snapshots) and any memory/ file.

Working dir is the repo root; use Read/Grep/Glob freely.

ADVERSARIALLY VERIFY these proposed staleness fixes for ONE file: docs/BACKLOG.md.

For EACH proposed fix: (1) open docs/BACKLOG.md and confirm stale_text appears VERBATIM (tighten/expand it to an exact, UNIQUE match if needed ? this text will be applied by exact string match, so it must be perfect); (2) re-open the cited code yourself and confirm the claim is genuinely contradicted by the CURRENT code; (3) confirm corrected_text is accurate, preserves voice, and is minimal; (4) DROP it if it is subjective, an owner-gated decision, already-correct, or you cannot verify the evidence. Output ONLY the survivors (verdict confirm or revise), with stale_text guaranteed to be an exact unique substring of the file.

PROPOSED FIXES:

```
[  
  {  
    "file": "docs/BACKLOG.md",  
    "stale_text": "new `backend/eval/splits.py` with `load_split(manifest) -> (train_ids, eval_ids)` keyed by **match id** (not file MD5 ? re-encodes share a match but differ in `compute_video_id`) + `assert_disjoint(train_ids, eval_ids)` that raises on overlap; wire the guard into the training entrypoint (`distill_local`/`training`) and into `run_regression.py` (baseline computed strictly on the eval split). **Verification:** unit test (overlapping ids ? `assert_disjoint` raises) + integration (a deliberately-leaked manifest fails the training entrypoint).",  
    "corrected_text": "***[DONE ? pure-logic core, PR #171]** `backend/eval/splits.py` now exists with `load_split(manifest) -> (train_ids, eval_ids)` keyed by **match id** (not file MD5 ? re-encodes share a match but differ in `compute_video_id`) + `assert_disjoint(train_ids, eval_ids)` that raises on overlap, with `tests/test_splits.py` (overlapping ids ? `assert_disjoint` raises). **STILL DEFERRED:** wire the guard into the training entrypoint (`distill_local`/`training`) and into `run_regression.py` (baseline computed strictly on the eval split); add the `split: \"train\"|\"eval\"` field to the collector's `curate export` manifest; the integration test (a deliberately-leaked manifest fails the training entrypoint).",
```

```

    "why_stale": "This ? item presents `backend/eval/splits.py` with
`load_split`/`assert_disjoint` + the overlap unit test as future 'Concrete design' to build, but
they were built and merged in PR #171 (commit e79bbc6) ? the file `backend/eval/splits.py` is
present in the tree and `tests/test_splits.py` exists. Only the pipeline wiring
(training/run_regression) and the collector manifest `split` field remain genuinely unbuilt
(splits.py is not yet imported by any training/eval entrypoint), so the enforcement gap is still
real but the module/assert/unit-test are no longer pending.",
    "severity": "medium"
}
]

```

2. assistant (2026-06-19T15:52:08.864Z)

I'll verify this proposed fix against the actual code and the current state of docs/BACKLOG.md.

3. user (2026-06-19T15:52:09.979Z)

49:[Omitted long matching line]

4. user (2026-06-19T15:52:10.546Z)

backend\eval\splits.py

5. user (2026-06-19T15:52:10.765Z)

tests\test_splits.py

6. assistant (2026-06-19T15:52:13.124Z)

Both files exist. Let me read the BACKLOG.md line and the actual code files to verify the claim.

7. user (2026-06-19T15:52:14.573Z)

```

30      - ? **Real GPU $/video & latency** ? benchmark the three LLM-step options (offline
ActionFormer vs self-hosted Qwen2.5-VL-7B vs Gemini paid-tier) on the target VM before pricing
the product. **Revisit:** when choosing the production LLM step.
31
32      ## Detector swap follow-ups
33      *(source: PR #8, ADR-005)*
34
35      - ? **`supervision` ByteTrack deprecation** ? `sv.ByteTrack` is deprecated as of
supervision 0.28.0 and **removed in 0.30.0** ; `requirements.txt` is pinned `<0.30.0` as a
stopgap. Migrate to `ByteTrackTracker` from the separate `trackers` package before lifting the
cap. **Revisit:** before bumping supervision past 0.30.
36      - ? **Real-video validation of the swap** ? the torchvision + ByteTrack Stage-1 is unit-
tested with mocks but **not yet validated on actual footage** for detection/tracking quality.
(ADR-004 separately found player pixel-motion is a weak rally proxy on broadcast video, so
expect tuning.) **Revisit:** when a real test clip + the pipeline are run together.
37      - ? **`yolo_hybrid` naming** ? the registry key, `yolo_` config keys, and `yolo-
hybrid-` DB detector tags keep the "yolo" name for back-compat despite no YOLO remaining.
Optional rename. **Revisit:** if/when a clean break is worth the migration.
38
39      ## Golden-set subproject
40      *(source: `docs/GOLDEN_SET_IMPLEMENTATION.md`, `docs/GOLDEN_SET_VENDORING.md`)*
41
42      - ? **Annotation guideline** ? **DONE (2026-06-02):** authored as the Roora brief +
illustrated guide in the collector repo `docs/annotation/` ? rally CSV schema; ignore dead time
/ include every rally (incl. faults & lets) / shot = one contact; `ending_reason`
(forced/unforced-error taxonomy, shared across net-separated sports); singles & doubles for
badminton & pickleball.
43      - ? **Internal gold key** ? hand-annotate 5?10 licensed clips to near-perfect as the
vendor answer key. **Unblocked.** **Revisit:** anytime.
44      - ? **GS-3 ? `HumanVendorProvider`** ? concrete adapter for the chosen annotation vendor
(submit/poll/fetch + DPA/secure-link controls). **Vendor selected for the initial pilot: Roora**

```

(Bengaluru; iMerit / Taskmonk / TELUS remain the fallback shortlist). ****Revisit:**** once the Roora pilot confirms quality, then build the adapter.

45 - ? ****GS-4 oracle model**** ? `AutomatedProvider` interface is stubbed (this work) with a deterministic fake. Wire a **concrete** fast-tier auto-labeler later. ****Oracle rule:**** it MUST be a different model lineage than production, or the regression test is blind to shared failures. ****Revisit:**** when a production LLM step is chosen (pick the oracle's lineage to differ).

46 - ? ****GS-5 ? regression triggers**** ? wire `regress()` into CI (pre-merge gate), on-demand CLI (the "surprise" case), and a scheduled run; include a deliberate re-baseline path. ****Revisit:**** after GS-4.

47 - ? ****Vendor pilot (Roora)**** ? run the paid pilot with ****Roora**** as a ****single-vendor pilot first**** (not a multi-vendor bake-off): assess quality-per-rupee against the \$4 bar; escalate to the iMerit/Taskmonk fallback only if it misses. ****Revisit:**** the pilot doubles as calibration once the gold key exists.

48 - ? ****Phase-1 video collection**** ? gather the round-1 golden corpus per `docs/GOLDEN\_SET\_PHASE1\_VIDEOS.md`: ****static self-recorded**** footage (mirror deployment, not broadcast), 3 sports incl. singles & doubles, across the diversity matrix; ~12 clips/sport with ****? held out by match****. ****Unblocked.****

49 - ? ****Train/eval separation guardrails**** (was `DEFERRED\_HARDENING\_PLAN` #13; design preserved here on archival 2026-06-14) ? train & eval videos live in ****separate catalogs**** today, kept apart by convention only; nothing **enforces** that a video used to train the learned segmenter (`backend/eval/training.py::build\_training\_set` ? `distill\_local`) is absent from the eval/regression set, so leakage would silently inflate the headline metrics. ****Concrete design**** (from the archived DEFERRED plan §2): split unit = ****match, never clip**** (hold ~? of each sport as eval, per `GOLDEN\_SET\_PHASE1\_VIDEOS.md`); add a `split: "train"|"eval"` field to the collector's `curate export` manifest (cross-repo contract, ADR-002); new `backend/eval/splits.py` with `load\_split(manifest) -> (train\_ids, eval\_ids)` keyed by ****match id**** (not file MD5 ? re-encodes share a match but differ in `compute\_video\_id`) + `assert\_disjoint(train\_ids, eval\_ids)` that raises on overlap; wire the guard into the training entrypoint (`distill\_local`/`training`) and into `run\_regression.py` (baseline computed strictly on the eval split). ****Verification:**** unit test (overlapping ids ? `assert\_disjoint` raises) + integration (a deliberately-leaked manifest fails the training entrypoint). Full rationale/risks: [`archives/DEFERRED\_HARDENING\_PLAN-COMPLETED-2026-06-14.md`](archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md) §2. ****Revisit:**** dedicated design session with the user (needs a collector-side manifest change).

50

51 **## Recording best practices**

52 ***(source: `docs/VIDEO\_RECORDING\_GUIDELINES.md`)***

53

54 - ? ****Validate recording hypotheses**** ? empirically test which capture conditions (resolution, fps, camera angle/height, lighting) actually move precision/recall, then promote the winners into user-facing "how to record" guidance. Only the static-camera rule is currently evidence-backed (ADR-004). ****Revisit:**** once a controlled corpus + the eval loop can measure per-condition deltas.

55

56 **## Code & test organization**

57 ***(source: owner review comments on PR #35, 2026-06-07)***

58

59 - ? ****`tests/` reorganization to reduce fragmentation**** ? owner wants to isolate test components on a grouping

60 metric (by subsystem/component) so `tests/` is less fragmented, mirroring the new `backend/pipeline/audio/` +

61 `backend/eval/audio/` layout (e.g. audio tests grouped under `tests/audio/`, eval tests under `tests/eval/`).

62 The exact grouping ****metric is the owner's design choice**** (consolidate small files vs. add per-area dirs ? the

63 "lesser fragmentation" goal slightly tensions with adding dirs), so this is left as a ****dedicated refactor PR****,

64 not guessed here. ****Revisit:**** when the owner specifies the grouping scheme. (The `backend/eval/audio/` half of

65 the request ? comment 1 ? was done in PR #35.)

66

67 **## Commercial Readiness & Low-Regret Moves ? Approved Decisions Checkpoint (2026-06-07)**

68

69 ***(source: Commercial Readiness Review + structure/sequencing discussion with Grok as second-in-command)***

8. user (2026-06-19T15:52:14.623Z)

```
1      """Train/eval split guard (pure-logic core, in-container).
2
3      Spec from BACKLOG.md "Train/eval separation guardrails" / DEFERRED #13
4      and plan M0:
5      - Split unit = match (never clip / file MD5).
6      - load_split(manifest) -> (train_ids, eval_ids) keyed by match id.
7      - assert_disjoint(train_ids, eval_ids) raises on any overlap.
8      - This would have caught train/eval leakage in training/eval paths.
9      - Cross-repo collector manifest `split` field wiring + wiring into
10         distill_local / run_regression deferred to later PR.
11
12      Usage (once wired):
13         train, eval_ = load_split(manifest)
14         assert_disjoint(train, eval_)
15      """
16      from __future__ import annotations
17
18      import json
19      import os
20      from typing import Any, Dict, Iterable, List, Set, Tuple, Union, cast
21
22
23      def load_split(manifest: Union[str, Iterable[Dict[str, Any]], os.PathLike[str]]) ->
24      Tuple[Set[str], Set[str]]:
25          """Load (train_ids, eval_ids) sets keyed by match id.
26
27          manifest: path to JSON (str/bytes/PathLike) or iterable of dicts. Each entry should
28          have 'match_id'
29          (or 'id' fallback; must be non-empty str for real match key; name/video_id
30          deliberately not used as they are weak/Md5).
31
32          Raises ValueError on:
33          - non-list manifest after load
34          - no real match-level key (empty/falsy after explicit check)
35          - split label not exactly "train" or "eval"
36
37          The collector 'split' field and cross-repo wiring is deferred; this supports
38          synthetic or pre-split manifests for the pure-logic guard and tests.
39          """
40          if isinstance(manifest, (str, bytes, os.PathLike)):
41              with open(manifest, "r", encoding="utf-8-sig") as f:
42                  raw = json.load(f)
43                  if not isinstance(raw, list):
44                      raise ValueError(f"manifest must be a list of entries, got {type(raw)}")
45                  data = cast(List[Dict[str, Any]], raw)
46          else:
47              if isinstance(manifest, dict):
48                  raise ValueError(f"manifest must be a list of entries, got
49                  {type(manifest)}")
50              data = list(manifest)
51              if not isinstance(data, list):
52                  raise ValueError(f"manifest must be a list of entries, got {type(data)}")
53
54          train_ids: Set[str] = set()
55          eval_ids: Set[str] = set()
56
57          for entry in data:
58              if not isinstance(entry, dict):
59                  continue
60              # Explicit not-None + non-empty for real match key; no silent fallback to
61              name/video_id (weak/Md5)
62              mid = entry.get("match_id")
63              if mid is None or str(mid).strip() == "":
```

```

59         mid = entry.get("id")
60         if mid is None or str(mid).strip() == "":
61             raise ValueError(f"no real match-level key (match_id or id) in entry:
{entry}")
62         mid = str(mid).strip()
63         sp = str(entry.get("split", "")).strip().lower()
64         if sp not in ("train", "eval"):
65             raise ValueError(f"split label must be exactly 'train' or 'eval', got {sp!r}
in entry: {entry}")
66         if sp == "eval":
67             eval_ids.add(mid)
68         else:
69             train_ids.add(mid)
70
71     return train_ids, eval_ids
72
73
74 def assert_disjoint(train_ids: Iterable[str], eval_ids: Iterable[str]) -> None:
75     """Raise if any match-id overlap between train and eval sets.
76
77     This would have caught train/eval leakage (e.g. a video used for both
78     training a segmenter and for regression/eval baselines).
79     """
80     t = set(str(x) for x in train_ids)
81     e = set(str(x) for x in eval_ids)
82     overlap = t & e
83     if overlap:
84         raise ValueError(
85             f"Train/eval leakage detected on match ids: {sorted(overlap)}. "
86             "This would have caught train/eval leakage."
87         )
88

```

9. assistant (2026-06-19T15:52:19.274Z)

The code confirms `splits.py` exists with `load\_split` and `assert\_disjoint`. Now let me verify the test file and check whether `splits.py` is actually wired into any training/eval entrypoint (to confirm "STILL DEFERRED" is accurate). Let me also verify the PR number and commit claims, and check `run\_regression.py` / `training.py` imports.

10. user (2026-06-19T15:52:19.711Z)

```

1     """Tests for backend/eval/splits.py (M0 train/eval guard core).
2
3     Per plan M0 + BACKLOG/DEFERRED #13 spec:
4     - assert_disjoint raises on overlap (match-id keyed, not MD5).
5     - disjoint passes.
6     - "this would have caught train/eval leakage" comment.
7
8     Cross-repo collector manifest wiring + entrypoint integration deferred.
9     """
10    import pytest
11
12    from backend.eval import splits
13
14
15    def test_load_split_from_list_and_assert_disjoint():
16        """Basic load + happy path for disjoint (match ids)."""
17        manifest = [
18            {"match_id": "match-s1", "split": "train", "name": "video1"},
19            {"match_id": "match-s2", "split": "train"},
20            {"match_id": "match-s3", "split": "eval"},
21            {"id": "match-s4", "split": "eval"}, # fallback id
22        ]
23        train, eval_ = splits.load_split(manifest)

```

```

24     assert "match-s1" in train and "match-s2" in train
25     assert "match-s3" in eval_ and "match-s4" in eval_
26     # should not raise
27     splits.assert_disjoint(train, eval_)
28
29
30     def test_assert_disjoint_raises_on_overlap():
31         """Overlapping match ids must raise with leakage message."""
32         train = ["match-a", "match-b"]
33         eval_ = ["match-b", "match-c"]
34         with pytest.raises(ValueError, match="leakage"):
35             splits.assert_disjoint(train, eval_)
36         # message includes the comment intent
37         with pytest.raises(ValueError, match="This would have caught train/eval leakage"):
38             splits.assert_disjoint(train, eval_)
39
40
41     def test_assert_disjoint_passes_disjoint_match_ids():
42         """Disjoint sets (including non-MD5 match ids) pass silently."""
43         splits.assert_disjoint(["match-001", "match-002"], ["match-003"])
44         splits.assert_disjoint(set(), ["m1"]) # empty train ok
45         splits.assert_disjoint(["m1"], set()) # empty eval ok
46
47
48     def test_load_split_from_path_like(tmp_path):
49         """load_split accepts path (for future manifest support)."""
50         p = tmp_path / "manifest.json"
51         p.write_text(
52             '[{"match_id": "m-train", "split": "train"}, '
53             '{"match_id": "m-eval", "split": "eval"}]'
54         )
55         train, eval_ = splits.load_split(str(p))
56         assert "m-train" in train
57         assert "m-eval" in eval_
58         splits.assert_disjoint(train, eval_)
59
60
61     def test_load_split_non_list_raises(tmp_path):
62         """Non-list manifest (object or other) must raise loud (would have caught bad
manifest)."""
63         p = tmp_path / "bad.json"
64         p.write_text('{"matches": [{"match_id": "m1", "split": "train"}]}')
65         with pytest.raises(ValueError, match="must be a list"):
66             splits.load_split(str(p))
67         # also for direct non-list
68         with pytest.raises(ValueError, match="must be a list"):
69             splits.load_split({"foo": "bar"})
70
71
72     def test_assert_disjoint_same_match_id_different_video_id_raises():
73         """Same match_id (different video_id e.g. re-encode) train vs eval must raise (core
leakage catch; this would have caught train/eval leakage)."""
74         manifest = [
75             {"match_id": "match-xyz", "split": "train", "video_id": "abc123"}, # video_id
is MD5, ignored
76             {"match_id": "match-xyz", "split": "eval", "video_id": "def456"},
77         ]
78         train, eval_ = splits.load_split(manifest)
79         with pytest.raises(ValueError, match="leakage"):
80             splits.assert_disjoint(train, eval_)
81
82
83     def test_load_split_and_assert_disjoint_end_to_end_leaked_manifest_raises(tmp_path):
84         """End-to-end: leaked manifest (same match_id train+eval) via load_split ->
assert_disjoint raises (this would have caught train/eval leakage)."""

```

```

85         p = tmp_path / "leaked.json"
86         p.write_text('{"match_id": "m-leak", "split": "train"}, {"match_id": "m-leak",
"split": "eval"}]')
87         train, eval_ = splits.load_split(str(p))
88         with pytest.raises(ValueError, match="leakage"):
89             splits.assert_disjoint(train, eval_)
90
91
92     def test_load_split_falsy_or_empty_id_raises():
93         """Empty/falsy match_id or id must raise (no silent drop/collision)."""
94         with pytest.raises(ValueError, match="no real match-level key"):
95             splits.load_split([{"match_id": "", "split": "train"}])
96         with pytest.raises(ValueError, match="no real match-level key"):
97             splits.load_split([{"id": None, "split": "eval"}])
98
99
100    def test_load_split_strict_split_label_raises():
101        """Split label must be exactly train/eval; typo or other raises (would have caught
mislabel masking leakage)."""
102        with pytest.raises(ValueError, match="split label must be exactly"):
103            splits.load_split([{"match_id": "m1", "split": "foo"}])
104        with pytest.raises(ValueError, match="split label must be exactly"):
105            splits.load_split([{"match_id": "m1", "split": "evaluation"}])
106

```

11. user (2026-06-19T15:52:20.716Z)

Found 8 files

```

backend\eval\rally_seg_eval.py
backend\eval\tolerance_metrics.py
backend\eval\segmentation_metrics.py
backend\eval\splits.py
backend\eval\eval_stats.py
backend\pipeline\detectors\fusion_features.py
backend\eval\calibration.py
backend\eval\gemini_refine.py

```

12. assistant (2026-06-19T15:52:25.798Z)

The grep confirms `splits.py` matches only within `backend/eval/splits.py` itself among the eval/backend files ? meaning no training entrypoint or `run\_regression.py` imports it. Let me confirm those two specific entrypoints don't reference splits, and verify the surrounding files exist.

13. user (2026-06-19T15:52:26.930Z)

No matches found

14. user (2026-06-19T15:52:27.899Z)

No files found

15. user (2026-06-19T15:52:28.232Z)

```

run_regression.py
.claude\worktrees\bhi-execpolicy\run_regression.py
.claude\worktrees\agent-a771ff1d85e18f2ed\run_regression.py
.claude\worktrees\tier1-windowing\run_regression.py
.claude\worktrees\w2-safeguard\run_regression.py
.claude\worktrees\hardchop\run_regression.py
.claude\worktrees\boxhill17\run_regression.py
.claude\worktrees\pr-stitch\run_regression.py
.claude\worktrees\gap-doc\run_regression.py
.claude\worktrees\resweep\run_regression.py

```

.claude\worktrees\recdoc\run_regression.py

16. assistant (2026-06-19T15:52:31.356Z)

Confirmed: neither `backend/eval/training.py` nor `run\_regression.py` import or reference `splits`. The wiring is genuinely still deferred, and the module + assert + unit test are genuinely built. Now let me verify the PR/commit attribution claim in the corrected_text. Let me check git history for `splits.py`.

17. user (2026-06-19T15:52:33.311Z)

c8ae56f address re-review for #171: fix direct-dict non-list guard hole (list(dict) keys bypass in else branch before listify); explicit dict raise matching 'must be a list' test msg. Verified smoke (direct now raises, paths preserved, happy/strict/leakage OK). plan append-only re-review entry + gate note. (per owner re-review comment 4702451231; fixes the CI red test_load_split_non_list_raises). All green where executable in container (owner re-runs full). No self-merge.

e1b6e68 address review for #171: non-list raise+check; explicit match key raise (no name/video fallback); strict train/eval split; pathlib+utf8sig; added leakage tests (same match diff vid, end-to-end, non-list, falsy, bad label) with comments. plan append-only review entry. (fixes 1-4 must, 5-6 nice)

ebdf331 fix: CI for PR171 - wrap long docstring lines (E501), use cast for json.load mypy compat, ensure files end with newline

e79bbc6 feat: M0 train/eval guard core - splits.py assert_disjoint + load_split + tests/test_splits.py (per plan M0 + BACKLOG/DEFERRED #13)

18. user (2026-06-19T15:52:34.960Z)

e79bbc6 feat: M0 train/eval guard core - splits.py assert_disjoint + load_split + tests/test_splits.py (per plan M0 + BACKLOG/DEFERRED #13)

34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding (#200)

e248d2d feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding

600d896 Merge pull request #171 from avidullu/feat/m0-splits-guard

8ba03e6 address review for #171: fix test_load_split_strict_split_label_raises (changed 'Eval' to 'foo' so genuinely invalid after .lower()); per owner re-review 4702490497, kept case-insensitive). Rebased on master fae8232.

c8ae56f address re-review for #171: fix direct-dict non-list guard hole (list(dict) keys bypass in else branch before listify); explicit dict raise matching 'must be a list' test msg. Verified smoke (direct now raises, paths preserved, happy/strict/leakage OK). plan append-only re-review entry + gate note. (per owner re-review comment 4702451231; fixes the CI red test_load_split_non_list_raises). All green where executable in container (owner re-runs full). No self-merge.

e1b6e68 address review for #171: non-list raise+check; explicit match key raise (no name/video fallback); strict train/eval split; pathlib+utf8sig; added leakage tests (same match diff vid, end-to-end, non-list, falsy, bad label) with comments. plan append-only review entry. (fixes 1-4 must, 5-6 nice)

ebdf331 fix: CI for PR171 - wrap long docstring lines (E501), use cast for json.load mypy compat, ensure files end with newline

--- PR171 in splits commit ---

8ba03e6 address review for #171: fix test_load_split_strict_split_label_raises (changed 'Eval' to 'foo' so genuinely invalid after .lower()); per owner re-review 4702490497, kept case-insensitive). Rebased on master fae8232.

c8ae56f address re-review for #171: fix direct-dict non-list guard hole (list(dict) keys bypass in else branch before listify); explicit dict raise matching 'must be a list' test msg. Verified smoke (direct now raises, paths preserved, happy/strict/leakage OK). plan append-only re-review entry + gate note. (per owner re-review comment 4702451231; fixes the CI red test_load_split_non_list_raises). All green where executable in container (owner re-runs full). No self-merge.

e1b6e68 address review for #171: non-list raise+check; explicit match key raise (no name/video fallback); strict train/eval split; pathlib+utf8sig; added leakage tests (same match diff vid, end-to-end, non-list, falsy, bad label) with comments. plan append-only review entry. (fixes 1-4 must, 5-6 nice)

ebdf331 fix: CI for PR171 - wrap long docstring lines (E501), use cast for json.load mypy compat, ensure files end with newline
e79bbc6 feat: M0 train/eval guard core - splits.py assert_disjoint + load_split + tests/test_splits.py (per plan M0 + BACKLOG/DEFERRED #13)

19. assistant (2026-06-19T15:52:41.740Z)

All claims verified. Commit `e79bbc6` introduced `splits.py` + `tests/test\_splits.py`, and PR #171 (merge commit `600d896` "Merge pull request #171 from avidullu/feat/m0-splits-guard") merged it. The attribution in `corrected\_text` ("PR #171", "commit e79bbc6") is accurate.

Now I need to verify the `stale\_text` is an exact unique substring. The proposed `stale\_text` starts with "new `backend/eval/splits.py`" which appears in the middle of the line. Let me confirm uniqueness and check whether the corrected_text correctly preserves the surrounding structure. The proposed fix replaces a mid-line chunk, but the chunk spans from "new `backend/eval/splits.py`" through the end of the "***Verification:***" sentence. Let me verify the exact text boundaries.

The stale_text ends with: "...integration (a deliberately-leaked manifest fails the training entrypoint)." ? but in the file (line 49), after "fails the training entrypoint)." there's "Full rationale/risks: [...]". Let me confirm the stale_text is a clean unique substring.

20. user (2026-06-19T15:52:42.834Z)

docs\BACKLOG.md:1

Found 1 total occurrence across 1 file.

21. user (2026-06-19T15:52:43.328Z)

docs\BACKLOG.md:1

Found 1 total occurrence across 1 file.

22. assistant (2026-06-19T15:52:54.466Z)

The stale_text is a unique substring. Now let me carefully validate that the proposed `corrected\_text` is faithful ? specifically that the "STILL DEFERRED" portion accurately lists what remains undone, and that it preserves the file's voice.

One concern: the proposed corrected_text drops the `split: "train"|"eval"` manifest field detail from its original position and re-lists it under STILL DEFERRED. Let me verify the original line still has the manifest-field clause BEFORE the stale_text starts, so it isn't duplicated. Looking at line 49: before "new `backend/eval/splits.py`" the text reads "...add a `split: \"train\"|\"eval\"` field to the collector's `curate export` manifest (cross-repo contract, ADR-002); new `backend/eval/splits.py`...".

So the manifest-field clause appears BEFORE the stale_text and would remain in the file. The corrected_text re-adds "add the `split: \"train\"|\"eval\"` field to the collector's `curate export` manifest" under STILL DEFERRED ? this would **duplicate** that clause. That's a flaw in the corrected_text. The minimal/faithful fix should not re-introduce the manifest-field clause since it already survives earlier in the same sentence.

Let me re-read the exact surrounding text to be precise about what to revise.

The original line structure is:

```
- "...split unit = **match...**; add a `split:...` field to the collector's `curate export` manifest (cross-repo contract, ADR-002); **[STALE_TEXT STARTS]** new `backend/eval/splits.py` with ... + `assert_disjoint(...)` ...; wire the guard into the training entrypoint ... and into `run_regression.py` ... **Verification:** unit test (...) + integration (...). **[STALE_TEXT ENDS]** Full rationale/risks: [...]"
```

So the corrected_text re-adds the manifest-field clause, creating a duplicate. I'll revise the corrected_text to remove that duplication while keeping the rest. This is a `revise` verdict.

23. user (2026-06-19T15:53:06.794Z)

Structured output provided successfully

24. assistant (2026-06-19T15:53:15.671Z)

Verified and submitted. Summary of my adversarial audit of the single proposed fix for
`C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md`:

****Verdict: REVISE (survives, with a corrected `corrected\_text`)****

Evidence confirmed:

- `stale\_text` appears verbatim and uniquely on line 49 of `docs/BACKLOG.md` (the ? "Train/eval separation guardrails" item).
- The cited module is genuinely built: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\\_splits.py` defines `load\_split` (match-id keyed, deliberately ignores `video\_id`/MD5) and `assert\_disjoint` (raises `ValueError` on overlap), and `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_splits.py` asserts overlapping ids raise. Git history confirms commit `e79bbc6` introduced both, merged via PR #171 (merge commit `600d896`).
- The "STILL DEFERRED" claim is also genuine: grep shows neither `backend/eval/training.py` nor `run\_regression.py` import or reference `\_splits`, so the enforcement wiring is truly unbuilt.

Why I revised rather than confirmed as-is: the proposed `corrected\_text` re-listed the `split: "train"|"eval"` collector-manifest-field clause under STILL DEFERRED, but that clause already survives ****earlier in the same sentence**** (before the `stale\_text` boundary). Re-adding it would duplicate it in the file. My revised text refers to it as "(above)" instead, keeping the edit minimal and duplication-free while preserving the document's voice.